



SMARTBOOK

A modern Accounting Management System for small and medium-sized businesses.

Built for the CodeMate Community

Contents

1. Project at a Glance
2. Problem, Users & Product Value
3. Customer Persona
4. Architecture & Technology
5. Core Accounting Workflow
6. Feature Map
7. Build: Design Mode
8. Build: Code Mode
9. Build Prompt Highlights
10. CORA: Planning & Implementation
11. CORA Task Archive
12. Implementation Milestones
13. Backend, Data & Integrations
14. Authentication & User Management
15. Responsive UI & Polish

01. PROJECT AT A GLANCE

SmartBook is a web-based **accounting management system** designed for small and medium-sized businesses. It brings customers, vendors, invoices, expenses, payments, reports, and company management into one centralized platform.

Core Modules

Module	Scope
Invoices & Billing	Create, manage and track customer invoices and recurring billing.
Customers & Vendors	Centralized management of client and supplier records with transaction history.
Payments	Record and track payments received from customers against specific invoices.
Expenses	Productivity tools for logging business-related spending and categorization.
Reports	Data-driven insights into revenue, expenses, and overall business health.
Company Profile	Management of business details, tax configurations, and branding settings.

02. PROBLEM, USERS & PRODUCT VALUE

The Problem

Small businesses often manage customers, invoices, expenses and payments across different tools. This silos data, leading to difficulty tracking pending payments and a lack of centralized financial information. SmartBook brings these operations together in one system.

Target Users

- **Administrator:** Manages company, users, customers, vendors, invoices and reports.
- **Staff / Accountant:** Creates invoices, manages expenses, records payments and views customers.

Product Value Proposition

The platform delivers value through a cohesive accounting strategy:

- Centralized accounting workflow and Easy customer/vendor management.
- Invoice and payment tracking with real-time expense management.
- Financial reporting and role-based access for secure collaboration.

03. CUSTOMER PERSONA

Primary Persona: Small Business Owner

- **Goal:** Manage the company's complete financial activity from one place, ensuring end-to-end visibility across revenue streams, outgoing expenses, and client transactions.
- **Needs:** A quick, real-time overview of business finances, streamlined invoice tracking, and effortless expense management to maintain cash flow health and simplify bookkeeping.
- **Pain Points:** Dealing with fragmented accounting data spread across multiple tools and experiencing significant difficulty tracking pending payments, which leads to administrative overhead and delayed cash flow.

SmartBook Solution

Provides one centralized dashboard where the administrator can manage, track, and monitor the company's accounting operations efficiently. By consolidating customer profiles, vendor records, open invoices, and expense logs into a single unified platform, SmartBook resolves data fragmentation and simplifies the tracking of outstanding balances.

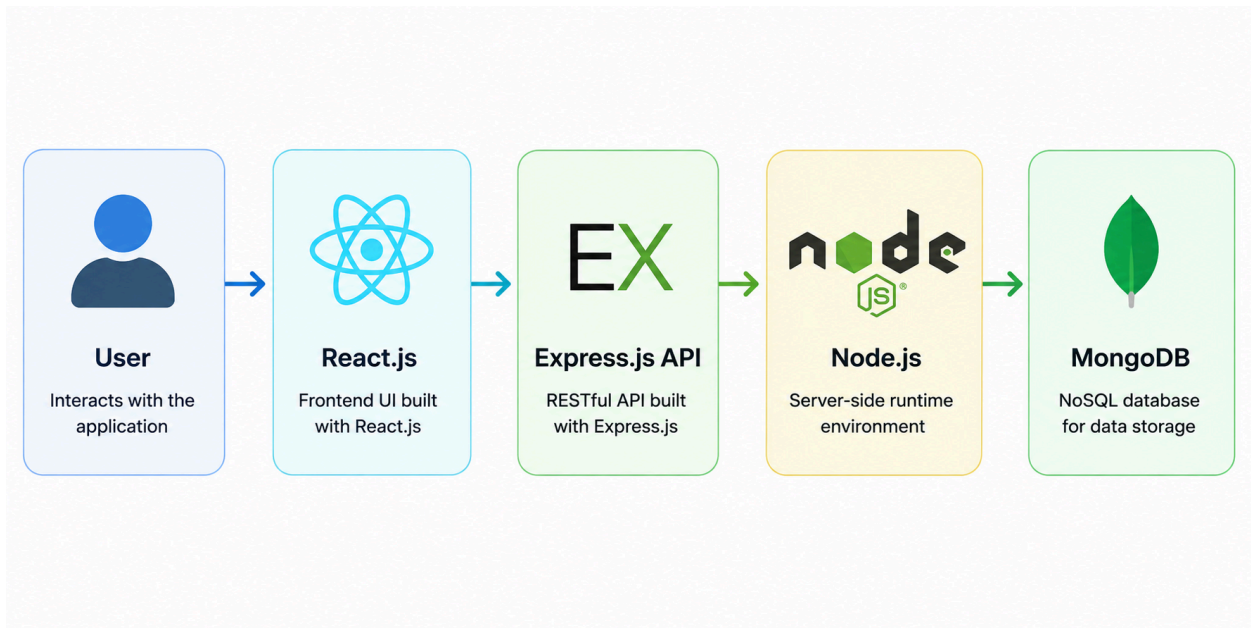
04. ARCHITECTURE & TECHNOLOGY

SmartBook utilizes a modern frontend-backend architecture designed for reliable accounting operations.

Technology Stack

Layer	Technology
Frontend	React.js, Tailwind CSS, React Router
Backend	Node.js, Express.js (REST API)
Database	MongoDB
Authentication	Custom JWT Authentication, Clerk

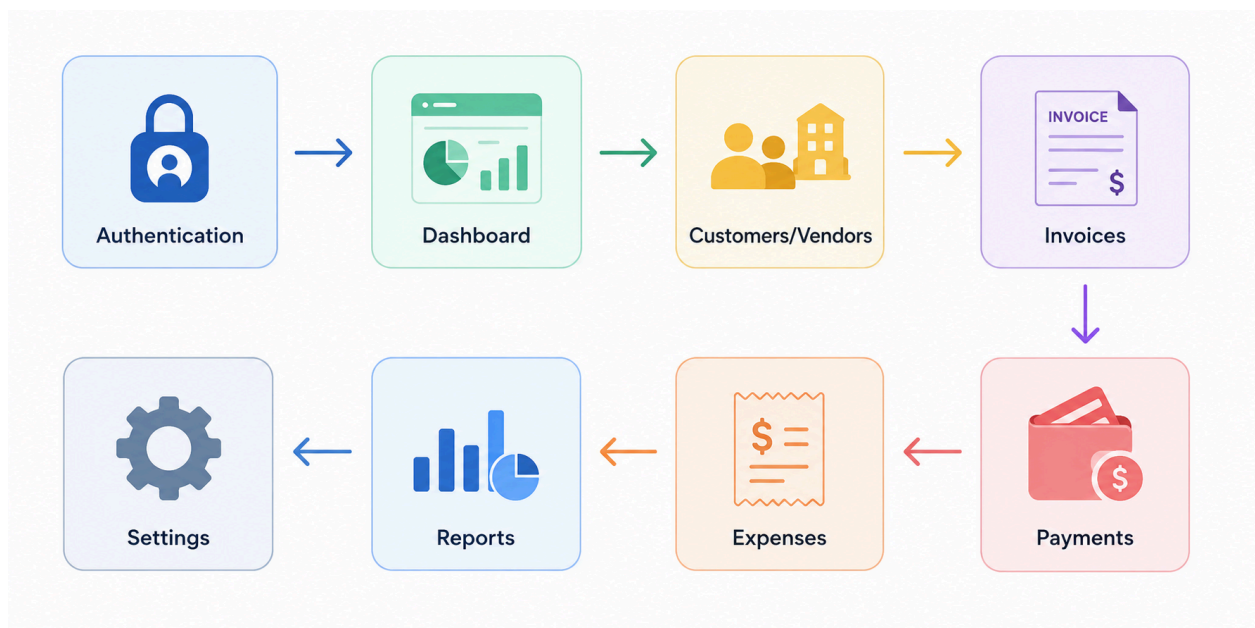
Architecture Flow



05. CORE ACCOUNTING WORKFLOW

The main SmartBook workflow connects core accounting operations in one platform, ensuring data consistency across the business lifecycle.

Operational Lifecycle



06. FEATURE MAP

Feature	Description
Dashboard	Overview of revenue, expenses, pending payments, and recent transactions.
Customers	Detailed management of lead contact information and transaction history.
Vendors	Management of supplier information and business-related records.
Invoices	Creation and tracking of customer invoices through different payment stages.
Payments	Logging of incoming funds and reconciliation against open invoices.
Expenses	Categorization and storage of all business spending activities.
Reports	Financial insights into business performance and accounting health.
Settings	Configuration for company profile, tax details, and member permissions.

07. BUILD: DESIGN MODE

SmartBook followed a professional design direction to bridge UI design and enterprise-style accounting requirements.

Objective & Direction

Create a minimal interface with white and gray surfaces and a blue primary accent.

- **Prompt 1 : SmartBook UI Design**

Design a modern, production-ready web application UI for an accounting software called “SmartBook”. The design should feel like a real SaaS product built by a professional product company rather than an AI-generated concept. Create a clean, minimal and professional accounting experience for small and medium-sized businesses, with a white background, subtle gray surfaces, blue primary accent, rounded components, soft shadows and spacious layouts.

- **Prompt 2 : Dashboard & Navigation**

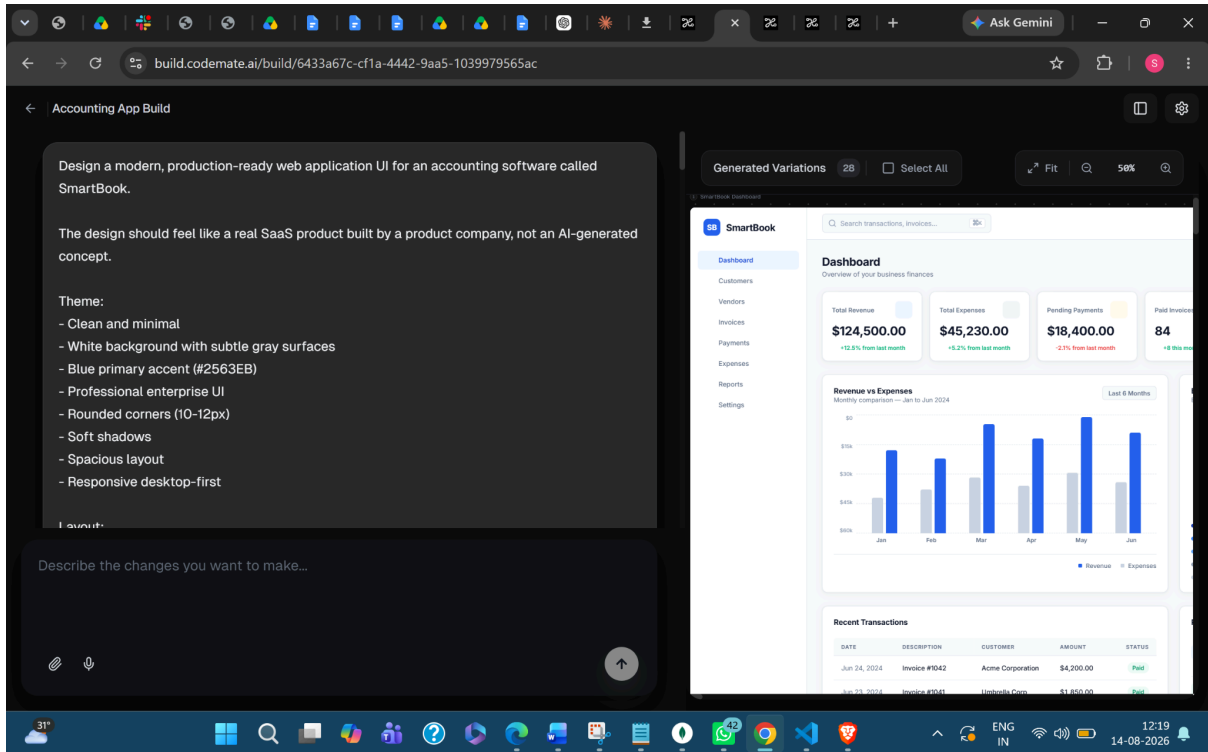
Design the SmartBook application with a responsive desktop-first layout and a collapsible left sidebar. Include Dashboard, Customers, Vendors, Invoices, Payments, Expenses, Reports and Settings in the main navigation. The dashboard should provide a clear overview of total revenue, total expenses, pending payments, paid invoices and active customers, along with revenue and expense charts and a recent transactions section. Keep the interface professional, structured and easy to scan.

- **Prompt 3 : Accounting Modules**

Extend the SmartBook interface into a complete accounting management experience. Create consistent pages for Customers, Vendors, Invoices, Payments, Expenses and Reports, with appropriate tables, forms, filters, actions and summary information. Maintain the same visual system across all modules and make the interface feel like one cohesive accounting product rather than a collection of separate pages.

- **Prompt 4 : Responsive & Visual Refinement**

Refine the SmartBook interface for a polished production-ready experience across desktop, tablet and smaller screens. Maintain the existing white and gray visual system, blue accent, professional typography and clean spacing. Ensure the sidebar, dashboard cards, tables, forms, charts and accounting modules adapt properly to different screen sizes. Avoid gradients, glassmorphism, neon colors and unnecessary visual effects.



08. BUILD: PROTOTYPE MODE

The Prototype Mode converted finalized designs into a functional React application with backend integration.

Implementation Prompts

- **Prompt 1 : Convert Finalized Design into Application**

The SmartBook UI design has been finalized. Do not redesign or change the existing layout, colors, typography, spacing, visual hierarchy or components. Now convert the approved design into a functional React.js application.

Implement the existing Dashboard, Customers, Vendors, Invoices, Payments, Expenses, Reports and Settings pages using reusable components, proper navigation and responsive layouts. Keep the approved SmartBook design as the source of truth throughout the implementation.

- **Prompt 2 : Accounting Functionality**

Continue developing the existing SmartBook application without changing the approved UI. Implement the core accounting workflows for customers, vendors, invoices, payments and expenses. Add the required forms, tables, actions, validation and state handling so users can create, view, update and manage accounting records through the application. Ensure the different modules remain connected and the overall experience feels like one complete accounting management system.

- **Prompt 3 : Backend & Database Integration**

The SmartBook frontend and UI are already implemented. Now connect the application with the backend and MongoDB database while keeping the existing frontend design unchanged. Set up the required API communication and database operations for users, customers, vendors, invoices, invoice items, expenses and payments. Replace static or mock data with persistent application data and make sure the main accounting workflows correctly save and retrieve records from the database.

- **Prompt 4 : Functional Iteration & Verification**

Review the current SmartBook implementation and verify the major application workflows end-to-end. Check dashboard data, customer and vendor records, invoice creation, payment tracking, expense management, forms, navigation and responsive behavior. Identify incomplete functionality or integration issues and implement the required corrections without changing the approved design.

build.codemate.ai/build/c6ef2438-dd74-4521-bdd9-fd842505868e

SmartBook Fronten...

ForkDownloadSettingsPreview site

SmartBook DashboardCustomers — Customer ManagementVendors — Vendor DirectoryInvoices — Invoice ListSmartBook — Landing Page

The UI design has been finalized. Do not redesign or change the layout, colors, typography, spacing, or components. Now start building the actual application using the existing design. Tech Stack: - React.js - Tailwind CSS - React Router - Node.js - Express.js - Neon PostgreSQL - Clerk Authentication Requirements: -

Show more (178 more words)

Data Storage Options

Choose where your project's data will be stored.

Save data online

Store information so it's accessible from anywhere

Skip for now

Build hiccup — your site is still live

Describe the changes you want to make

+🗑️⬆️

FilesPreviewhttps://heartfelt-maamout-cba245.codemate.build

Files

public

sitemap.xml

tsconfig.tsbuildinfo

tsconfig.json

README.md

postcss.config.mjs

package.json

next.config.ts

next-env.d.ts

lib

mongodb.ts

auth-store.ts

store.ts

data.ts

utils.ts

hooks

use-mobile.ts

components.json

components

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

27

28

See

<https://help.github.com/articles/ignoring-files/> for more about ignoring files.

dependencies

/node_modules

/.pnpm

/.pnpm/*

/.yarn/*

l.yarn/patches

l.yarn/plugins

l.yarn/releases

l.yarn/versions

testing

/coverage

next.js

/.next/

/out/

production

/build

misc

.DS_Store

*.pem

debug

npm-debug.log*

31°

🪟🔍📄🗂️👤🔗🔄📱📧📅🕒

ENG IN📶🔊🔋12:2314-08-2026🔔

09. BUILD PROMPT HIGHLIGHTS

Representative Prompts

- **Prompt 1: Product Design**

Design a modern, production-ready web application UI for an accounting software called “SmartBook”. Create a clean, professional and responsive SaaS experience with a white background, subtle gray surfaces, blue primary accent, spacious layouts and an enterprise-style visual language suitable for accounting software.

- **Prompt 2: Dashboard**

Create a SmartBook accounting dashboard that gives users an immediate overview of their business finances. Include total revenue, total expenses, pending payments, paid invoices and active customers, along with monthly revenue versus expenses, expense categories, recent transactions and recent activity. Keep the dashboard clean, readable and focused on useful financial information.

- **Prompt 3: Accounting Management**

Build the core SmartBook accounting modules for Customers, Vendors, Invoices, Payments, Expenses and Reports. Each module should provide the appropriate tables, forms, actions and information required to manage accounting records efficiently. Maintain consistent navigation, components, spacing and styling across the entire application.

- **Prompt 4: Functional Application**

Take the finalized SmartBook design and turn it into a complete functional accounting application. Preserve the existing UI while implementing reusable React components, navigation, forms, tables, accounting workflows, validation and responsive behavior. The final application should feel like a production-ready accounting product rather than a static prototype.

10. CORA: PLANNING & IMPLEMENTATION

CORA served as the backend layer, particularly rebuilding the authentication architecture and establishing reliable database connectivity.

Strategic Prompts

- **Prompt 1 : Authentication Rebuild**

Analyze the current SmartBook authentication implementation and create a structured plan to rebuild the authentication layer properly. Review the existing login, registration, logout, session handling, middleware and authentication-related files. Identify the required backend routes, user data handling, authentication logic and dependencies, and define the implementation steps needed to establish a reliable authentication flow without leaving broken or duplicated authentication logic.

- **Prompt 2 : MongoDB Authentication Integration**

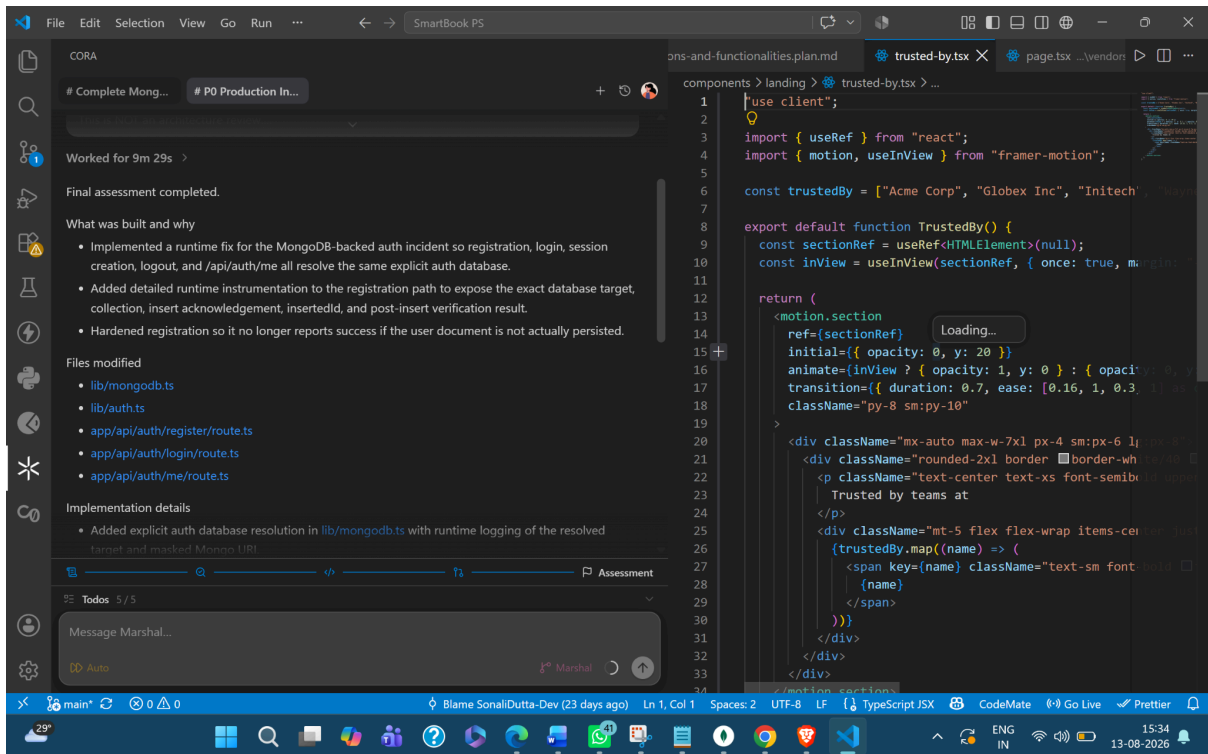
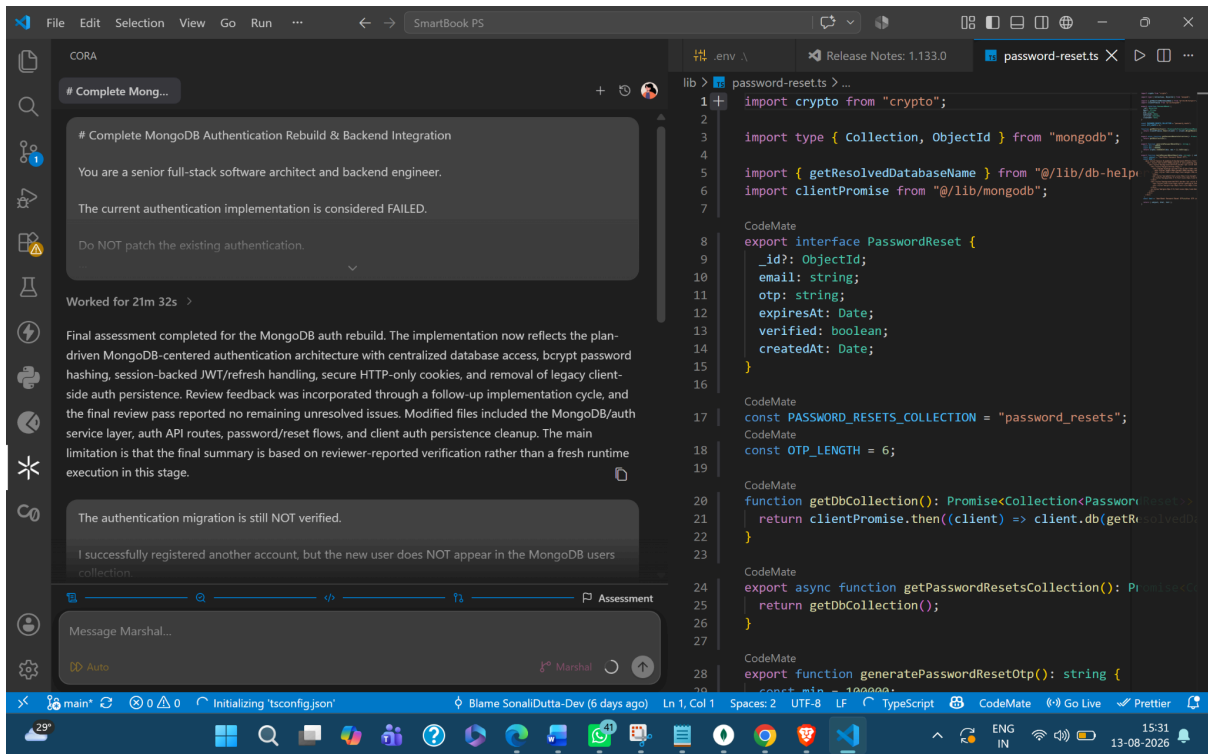
Rework the SmartBook authentication system so that it is properly connected to MongoDB and works reliably at runtime. Review the existing database connection, user model, authentication routes and session handling before making changes. Implement the required database operations, password handling, JWT-based authentication and protected route logic while keeping the existing application structure intact. Verify that authentication data is correctly persisted and retrieved from MongoDB.

- **Prompt 3 : Authentication Runtime Verification**

Review the implemented SmartBook authentication flow end-to-end and verify registration, login, logout, session validation and protected routes. Check the communication between the frontend authentication pages, API routes, middleware and MongoDB. Identify any runtime errors, incorrect session behavior or database connectivity issues, then implement the necessary corrections and re-test the complete authentication flow.

- **Prompt 4 : Final Authentication & Application Integration**

Verify that the rebuilt SmartBook authentication layer integrates correctly with the rest of the application. Ensure authenticated users can successfully move through the login flow and access the appropriate application pages while unauthenticated requests remain protected.



11. CORA TASK ARCHIVE

Category	Implementation Focus
Authentication	Implementation and verification of complete end-to-end authentication mechanisms, including user registration, login, logout, JWT auth, secure password hashing, refresh tokens, session validation, and protected routes.
Database	Configuration of MongoDB connection, establishing user and session collections, environment validation, and performing runtime database verification to ensure persistence.
Integration	Connecting frontend authentication pages directly with backend API middleware, ensuring proper route protection, session persistence, navigation post-auth, and clean API response handling without broken logic.

12. IMPLEMENTATION MILESTONES

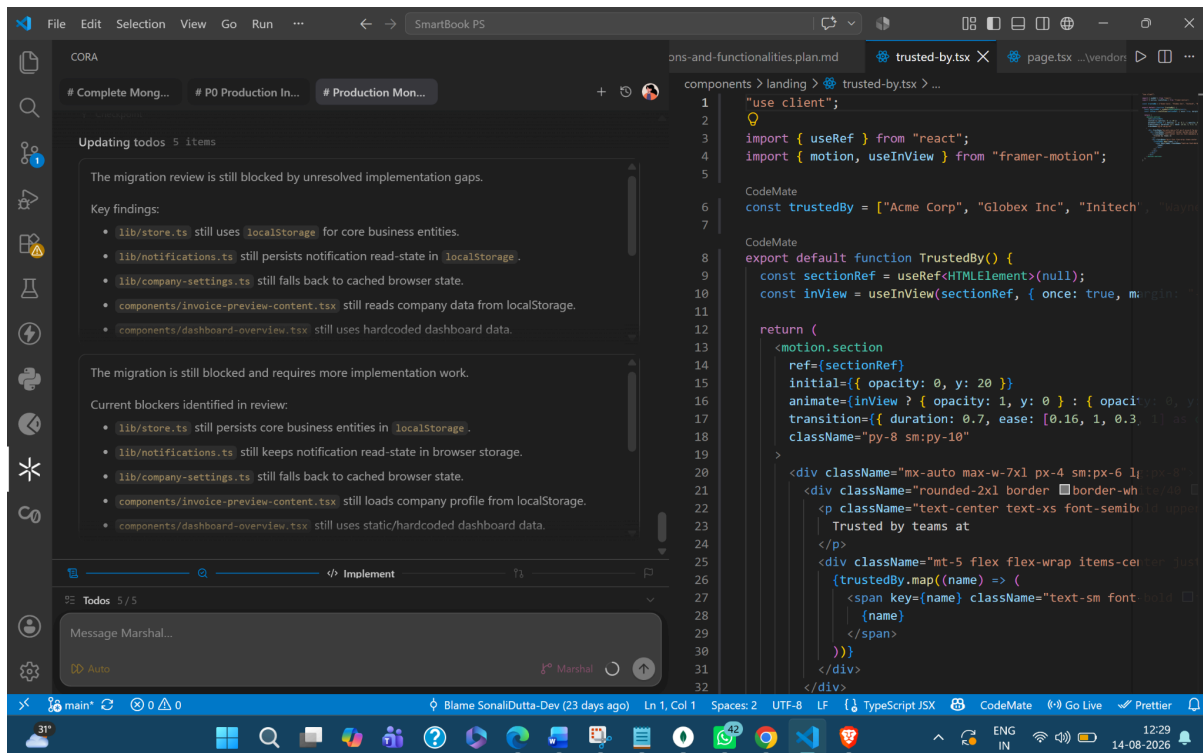
Phase	Milestone	Focus Area
1	Design	SmartBook UI creation using Build Design Mode.
2	Frontend	React application conversion with responsive layouts.
3	Backend	Express API and MongoDB database operations.
4	Authentication	JWT rebuild and role-based access verification.
5	Verification	End-to-end testing of core accounting workflows.

13. BACKEND, DATA & INTEGRATIONS

SmartBook leverages a modular Node.js/Express architecture to manage data models for Users, Companies, Invoices, and Payments. Strict database scoping ensures that accounting records are isolated per organization.

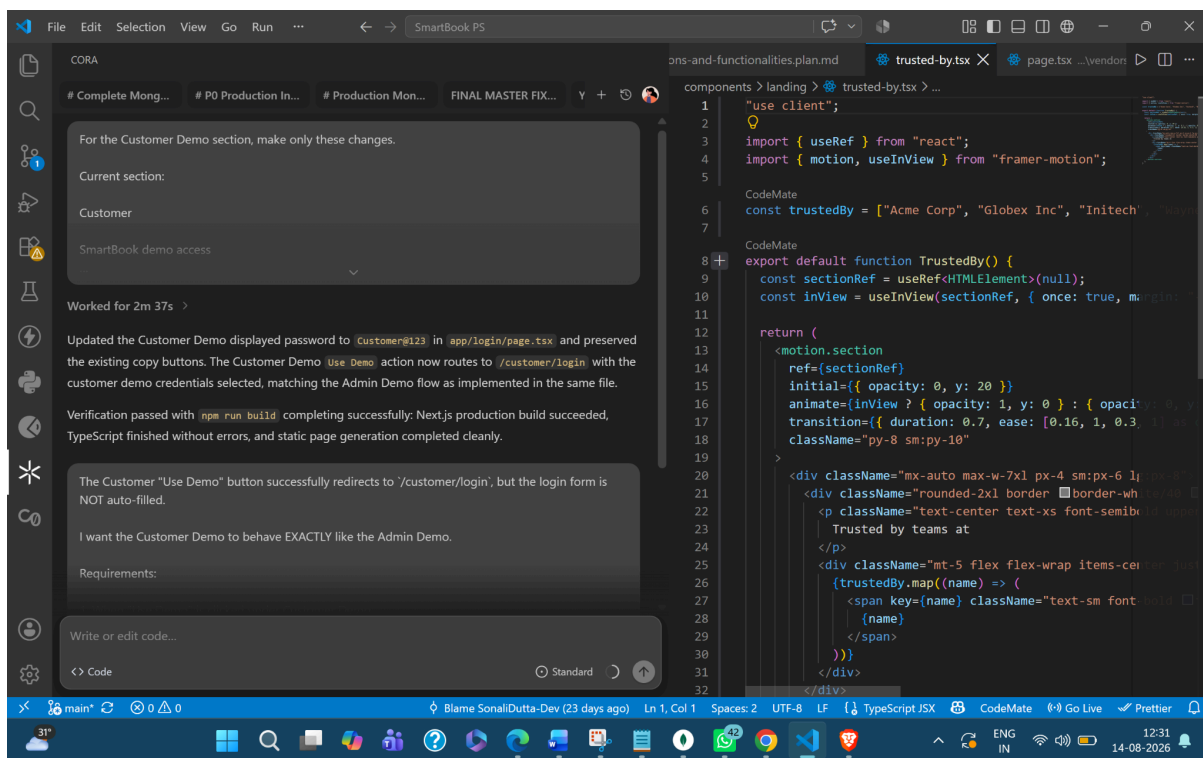
Core Models

- **Invoices:** Primary transaction record linked to customers and invoice items.
- **Expenses:** Categorized spending logs for business health monitoring.



14. AUTHENTICATION & USER MANAGEMENT

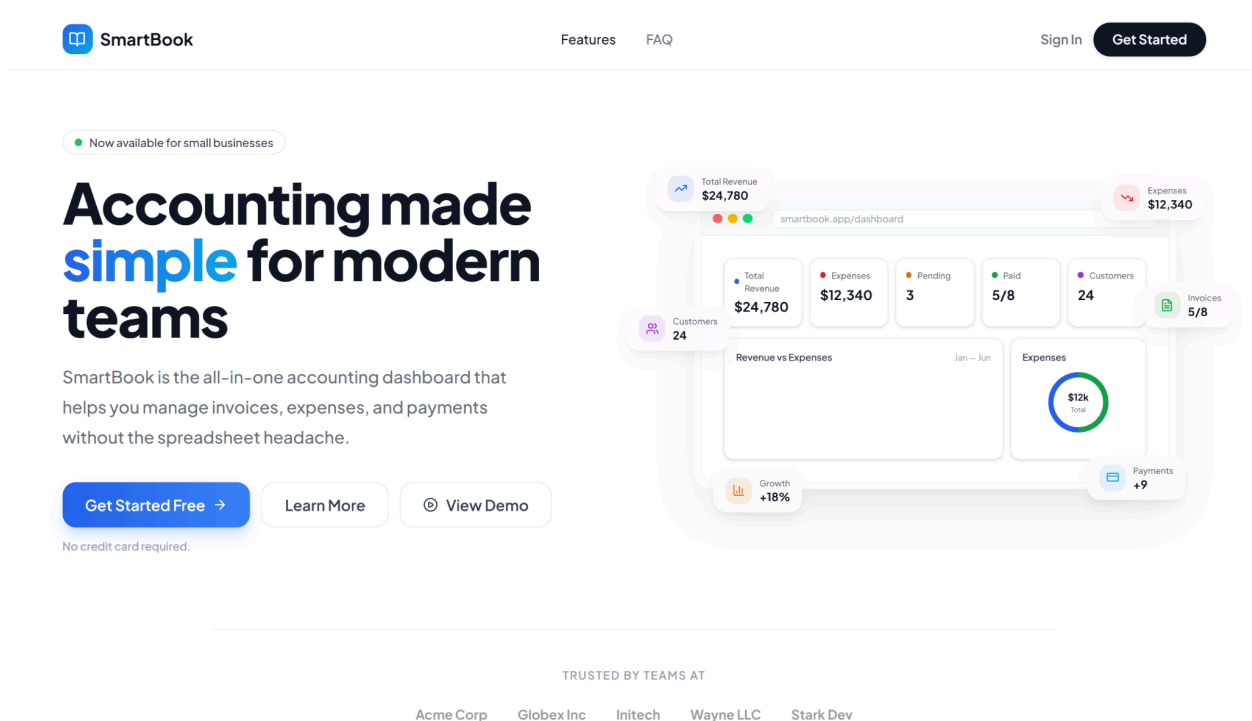
- **Authentication:** SmartBook uses Clerk Authentication with role-based access control to ensure data security.
- **Administrator Access:** Administrators are provided with full system visibility.
- **Staff & Accountant Access:** Staff and Accountants are restricted to operational tasks, including invoice creation and expense logging.



15. RESPONSIVE UI & POLISH

15.1 Desktop View

Features a full sidebar and multi-column tables for comprehensive accounting management.

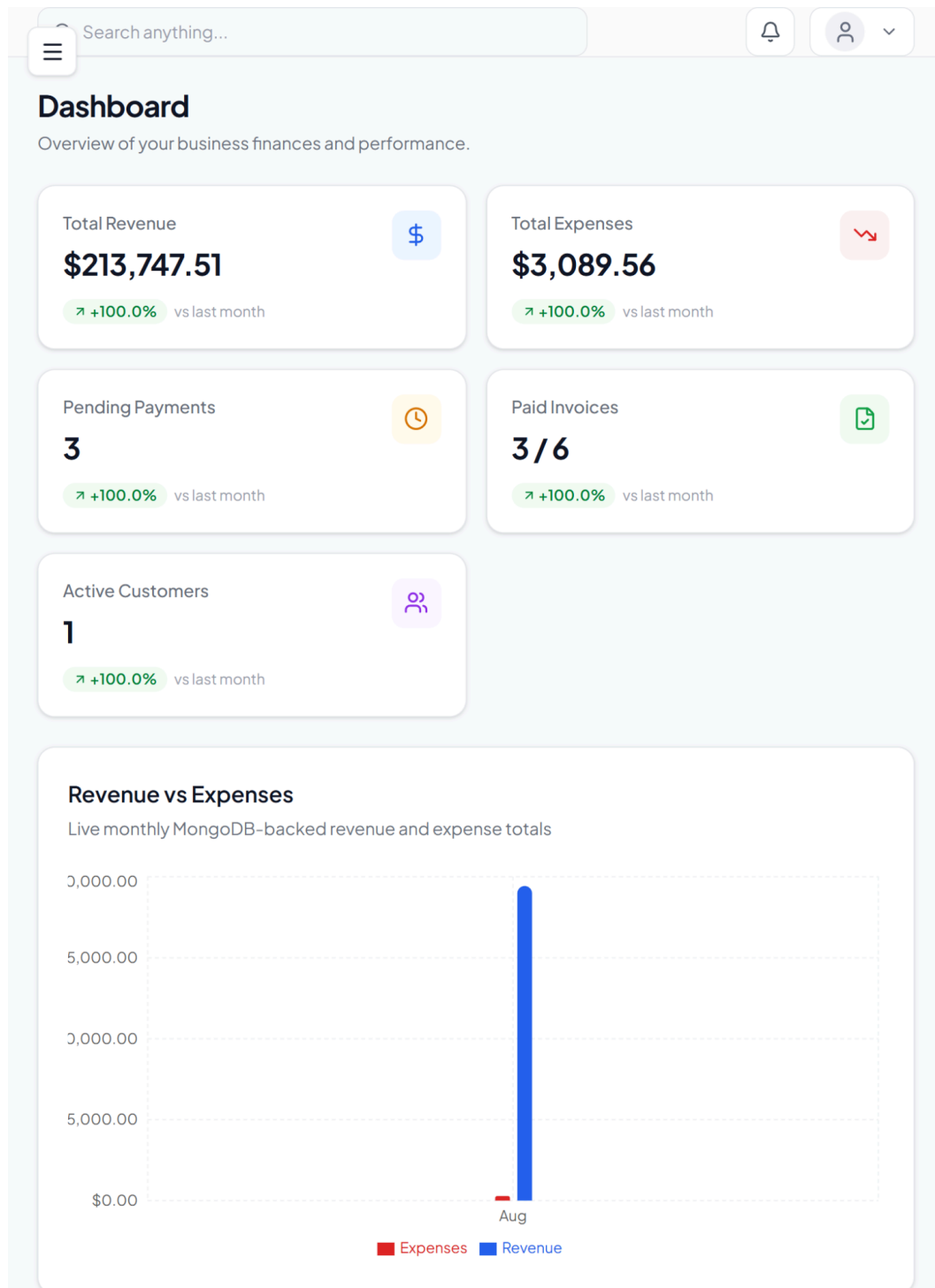


Everything you need to run your books

One dashboard for invoices, expenses, vendors, customers, and payments.

15.2 Tablet View

Adjusted navigation and content density for touch-friendly interfaces.



15.3 Mobile View

Stacked layouts and hamburger navigation for mobile accessibility.

